

Team Members: Bhagya Puppala, Cassandre Korvink, Janani Vakkanti, Muskan Khepar, Nathan Arimilli

[Project Presentation Slides](#) and [Blog Post](#) and [GitHub Page](#)

Abstract

Recommender systems struggle when users or items lack interaction history, especially across different content domains. This project investigates whether Large Language Model (LLM) embeddings can bridge that gap by mapping movies and books into a shared semantic space, enabling meaningful cold-start recommendations without shared users or cross-domain ratings. We construct rich semantic representations for each item by fusing plot summaries, review text, metadata, and rating cues, then embed these components using all-mpnet-base-v2, a sentence-level transformer model. Two complementary approaches are evaluated: a pure zero-shot method that performs retrieval directly in the fused embedding space, and a

lightweight neural mapping model trained on synthetic movie–book pairs generated from zero-shot neighbors.

Across system-level and user-level evaluations, LLM-driven semantics prove surprisingly powerful. Zero-shot retrieval alone successfully recovers relevant cross-domain matches, achieving strong ranking performance on synthetic alignment pairs. The learned mapping model trades some raw recall for improved structural alignment, producing category-consistent recommendations and performing well in a case study driven by a Nolan-style movie profile. Compared to traditional collaborative filtering, which cannot operate in cross-domain cold-start settings, our semantic pipeline offers a scalable and training-light alternative. Together, these results demonstrate that LLM-based representations can form an effective “semantic bridge” between media domains, opening the door to cold-start recommendation systems that rely on meaning rather than historical overlap.

Introduction & Background

From “What Should I Watch Next?” to “What Should I Read Next?”

Most real-world recommenders live inside a single domain:

Netflix suggests movies, Goodreads suggests books, Spotify suggests songs. But people don’t consume stories in isolation. A fan of dark, twisty sci-fi movies might also love cerebral science-fiction novels or graphic novels with similar tone and themes.

The problem we tackle in this project is **cross-domain cold-start recommendation**: Given only a user’s movie taste, can we recommend books they are likely to enjoy without any overlapping users, joint ratings, or shared IDs between the two domains?

This matters because cold-start users are the norm, not the exception. New users often arrive with rich preferences in one

medium (e.g., films they love) but no history in another (e.g., books on a new platform). Traditional collaborative filtering cannot help in this setting. If we can reliably map “movie taste → book suggestions” using only text and pre-trained language models, we get a reusable recipe for story-level discovery that generalizes beyond this specific movie→book example.

What is the “cold-start” problem, and why is our setting harder?

Most recommenders learn from patterns in past behavior:

“People who watched these movies also watched...”, “Users who bought this book also bought...”.

Mathematically, you can think of a big user-by-item table where each cell is a rating, click, or purchase.

The **cold-start problem** is what happens when that table has empty rows or columns:

User cold-start (new user): A brand-new user arrives with no ratings yet. The system has no row for them, so it cannot tell which existing users they are similar to. In practice, sites fall back to crude heuristics like “popular right now” or ask onboarding questions.

Item cold-start (new item): A new book or movie is added to the catalog, but nobody has interacted with it yet. The system has a column with no data, so it does not know who might like it.

Cross-domain cold-start (our project): This is a stricter version: the user has history, but in a different domain.

In our setting, the user has a rich movie history (they love Nolan-style sci-fi and thrillers) but **no book ratings at all** on the book platform. The movie site (Rotten Tomatoes) and the book site (Amazon Books) do not share user IDs, ratings, or a joint matrix, so a traditional collaborative filtering model on books literally sees this user as an empty row.

So the question we ask is:

Given only a user's **movie** taste, can we still recommend **books** that fit their preferences *without* shared users, joint ratings, or manually aligned IDs between the two domains?

That is what we mean by **cross-domain cold-start recommendation**: we are trying to make good recommendations when the usual “people who liked X also liked Y” signal does not exist, and the only bridge we have is **text** (plots, reviews, categories) fed into LLM embeddings.

Related Work: Collaborative Filtering, Content, and Semantic Embeddings

Classical collaborative filtering (CF) methods (matrix factorization, Neural Collaborative Filtering (NCF), etc.) learn latent factors from user–item interaction data. They work well within a single domain when there is enough rating density, but they break down when the user is new (no ratings yet), or the

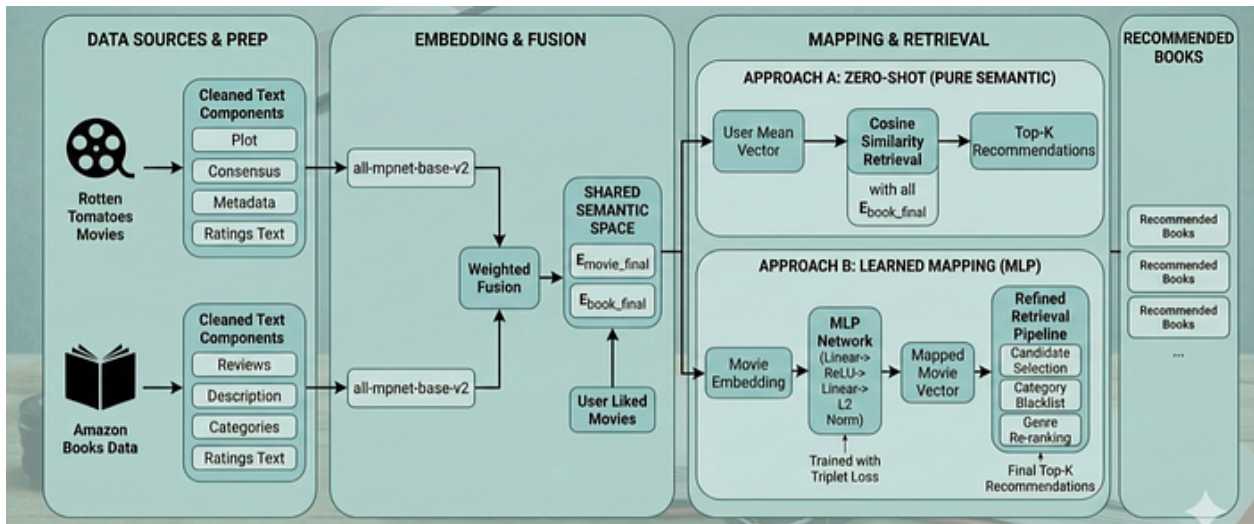
target domain is different (books vs movies) and has no shared users.

Content-based recommenders address cold-start by using item features (e.g., genres, tags, keywords, descriptions) to compute similarities. In practice, though, traditional bag-of-words and TF-IDF style features struggle to encode nuance: they miss tone, pacing, and thematic depth that matter a lot for storytelling media.

Recent advances in pre-trained language models and sentence embeddings offer a third path: represent items directly in a high-dimensional semantic space where cosine distance roughly reflects conceptual similarity. This has sparked work on semantic retrieval, zero-shot matching, and cross-modal / cross-domain recommendation. Our project builds on that trend by asking how far we can get with off-the-shelf LLM embeddings plus careful representation engineering, and what is gained (or lost) by adding a learned alignment layer.

Approach Overview: Building a Semantic Bridge Between Movies and Books

Press enter or click to view image in full size



At a high level, our pipeline has four stages:

1. Build a shared semantic space for movies and books.

We start from large, text-heavy datasets: Rotten Tomatoes for movies and Amazon Books metadata + reviews for books. We construct “semantic bridge” text fields (plots, critic consensus, categories, and especially user reviews) and embed them with a

SentenceTransformer (all-mpnet-base-v2). For each item, we fuse multiple components (plot, metadata, ratings text, review aggregates, etc.) into a single normalized vector. This gives us E_movie_final and E_book_final in a common embedding space.

2. Anchor expectations with an in-domain collaborative baseline.

We train a book-only NCF model on real user–book ratings. This provides a warm-start benchmark for what a traditional rating-based recommender can do within the book domain, and highlights that it cannot solve the movie-only cold-start scenario.

3. Cross-domain semantic recommenders: two approaches.

Approach A (Zero-Shot Semantic Retrieval) uses the fused embeddings directly. Represent a user by averaging the

embeddings of movies they like, then retrieve nearest-neighbor books by cosine similarity. No extra training; this is our “pure semantic” baseline.

Approach B (Learned Mapping with Triplet Loss) trains a small MLP to map movie embeddings into book space using synthetic movie–book pairs constructed from zero-shot neighbors. The mapping is trained with a contrastive (triplet) objective so that mapped movies sit closer to their matched books than to random negatives.

4. User-level retrieval and evaluation.

At inference time, we build user taste vectors from liked movies, run movie→book retrieval with either the zero-shot or mapped embeddings, and optionally apply genre- and category-aware re-ranking. We evaluate along three axes: in-domain book NCF (how well ratings-only CF does on books), system-level movie→book retrieval on synthetic alignment pairs (HitRate@K,

NDCG, genre alignment), and user-level case studies (e.g., a “Nolan-style” movie fan) with genre-overlap and rating analysis, plus visual diagnostics of the embedding space.

Contributions and What’s Novel in This Project

While the individual building blocks of SentenceTransformers, NCF, triplet loss are standard, the way we combine them is what makes this project distinctive. Our main contributions are:

A practical semantic bridge between movies and books without shared users. We show that LLM-based embeddings, when thoughtfully engineered and fused, are enough to support zero-shot movie→book recommendations in a realistic, large-catalog setting.

A clear comparison between zero-shot semantics and a learned cross-domain mapping. Instead of assuming “more training is better,” we empirically compare a pure zero-shot model and a triplet-trained mapping, showing where the

mapping helps (category alignment, controlled behavior) and where it hurts (recall on zero-shot-defined synthetic pairs).

An evaluation framework tailored to cross-domain

cold-start. We combine an in-domain NCF baseline on real user–book data, system-level movie→book retrieval on synthetic alignment pairs, and user-level case studies with genre and rating diagnostics. This gives a nuanced view of how semantic recommenders behave when traditional CF has no signal.

Representation-first design that foregrounds text and

user reviews. Rather than treating embeddings as a black box, we carefully design the text inputs, component embeddings, and fusion weights so that user reviews and narrative content carry most of the signal. This makes the resulting recommendations more interpretable and better aligned with story-level semantics instead of just surface keywords.

Together, these pieces support our central claim: LLM semantic embeddings can be used to construct a shared conceptual space for movies and books that enables cross-domain cold-start recommendation, even in the absence of shared users or joint training data. The rest of the report walks through how we built that space, how we probed it with baselines and mapping models, and what we learned from evaluating it at both system and user levels.

Data Collection and Description

Before we could build any cross-domain recommender, we first needed a rich, paired view of movies and books that actually had something to say about story, tone, and audience reaction. This phase focused on assembling that raw material.

Source Datasets

We draw from three publicly available CSV datasets, all sourced from Kaggle and designed for academic use:



Rotten Tomatoes Movies and Critic Reviews. File used: rotten_tomatoes_movies.csv. Each row represents a single movie, with columns for movie_title, movie_info (plot synopsis), critics_consensus, genres, directors, authors (for adaptations), tomatometer_rating, audience_rating, tomatometer_count, audience_count. This gives us a mix of narrative text (plot, consensus) and crowd signals (ratings and counts).

Amazon Books Metadata. File used: books_data.csv. Each row is a unique book title with Title, description, authors,

categories (subject categories/genres). This is our main source of structured book information and publisher descriptions.

Amazon Books Reviews and Ratings. File used:

Books_rating.csv. Each row corresponds to a single user review, with Title, review/text, review/summary, review/score (1–5), review/helpfulness (e.g., 3/5). This table is where we get the user generated narrative signal that drives most of our semantics.

All three datasets are large scale and text heavy, which is exactly what we need for LLM based embedding.

Relevant Characteristics of the Combined Data

Once we align schemas and join on Title, the data has a few key characteristics that shape the rest of the project:

Scale and granularity. The combined books data starts at roughly 3 million review rows, covering about 140k unique book titles. Each book can have many reviews, so the “book” side is

inherently multi-instance, while each movie is a single row with rich metadata.

Strong textual coverage. Movies come with plot summaries, critic consensus blurbs, genre strings and director names, and numeric ratings and counts. Books come with long form review text, short review summaries, category strings, descriptions when available, and per review star ratings and helpfulness scores.

User centric signals. The Amazon reviews table gives us detailed, opinionated text for each book, plus a review/score rating and a review/helpfulness ratio. These fields are crucial later when we decide which reviews to keep and how to weight them when aggregating into a single book vector.

Structured fields for alignment. Both domains expose category-like information: movie genres and book categories. These fields act as a bridge for genre aware synthetic pairing and

evaluation in later phases (for example, checking whether a thriller movie matches a thriller or mystery book).

Methods of Acquisition and Initial Assembly

The acquisition and assembly process followed a clear, reproducible pipeline:

Download and load CSVs. Each Kaggle dataset was downloaded as a CSV and loaded into a pandas DataFrame: books_df from books_data.csv, books_rating_df from Books_rating.csv, movies_df from rotten_tomatoes_movies.csv.

Column selection and schema trimming. We kept only the fields needed for semantics and basic popularity, dropping purely technical or UI oriented columns such as links, image URLs, and detailed critic breakdowns. For example, for movies we kept title, plot, consensus, genres, directors, authors, and rating/count fields, and dropped link, runtime, and production company fields. For books we kept title, description, authors,

categories in one table and review text, review summary, rating, and helpfulness in the other.

Book level merge on title. We merged book metadata and book reviews on Title to create a review level table:

```
books_combined = merge(books_rating_df, books_df, on="Title")
```

This gives us one row per review, with both review text and rating, and book level metadata (description, authors, categories).

Quality filtering around review text. We filtered out rows with missing review/text and kept everything else, because reviews are our primary semantic signal for books. Missing descriptions were not dropped; instead, they were filled with a neutral placeholder like “No description available” and treated carefully in later semantic construction.

ID assignment for stable referencing. For movies, we assigned a unique movie_id by resetting the index on the cleaned movies table. For books, we assigned a book_id per Title (one book id per unique title) and assigned a review_id per row to keep track of individual reviews. This dual ID structure lets us aggregate multiple reviews into a single book_id representation, and still trace back to the underlying review rows when needed.

By the end of these initial steps, we have a clean, linked, and ID stable dataset: movies with rich plot and rating fields, and books represented as many reviews tied back to a single title. This forms the raw substrate for the LLM embedding and fusion steps described in the next section of the report.

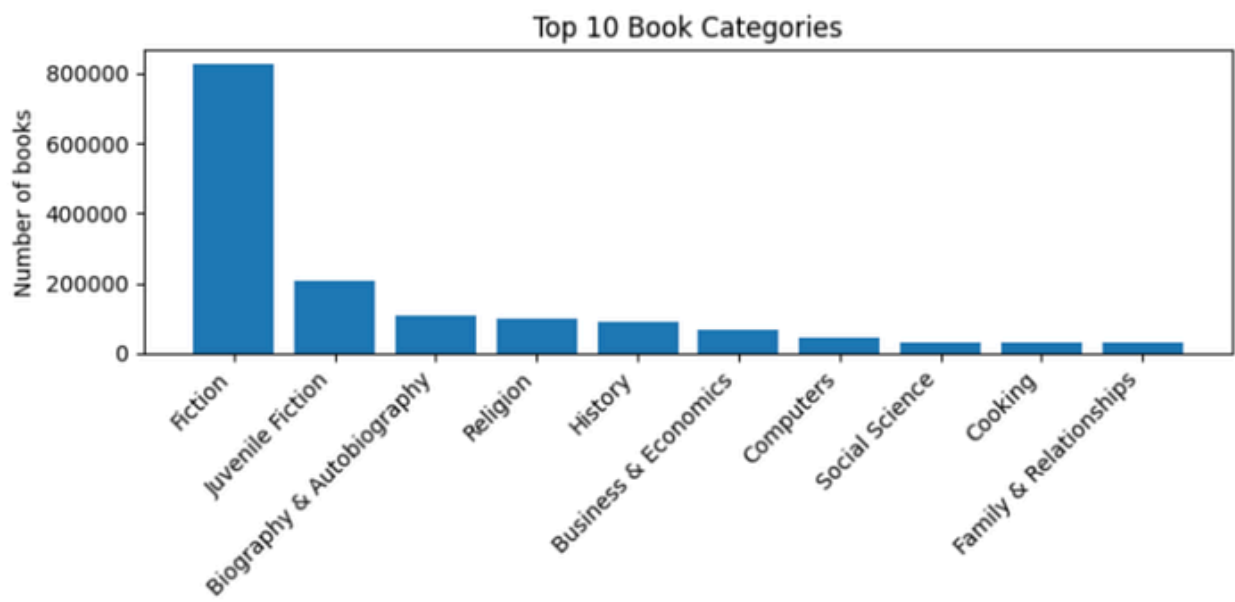
Data Pre-Processing & Exploration

Dataset Composition: What Movies and Books Do We Have?

Our book catalog is heavily skewed toward Fiction, with Juvenile Fiction a distant second, and smaller but still meaningful

volumes in Biography & Autobiography, Religion, History, Business & Economics, Computers, Social Science, Cooking, and Family & Relationships. This tells us two things: most recommendations will naturally live in broad fiction, but we also have enough depth in specific non-fiction verticals to support more niche cross-domain matches (e.g., documentaries → history/business books).

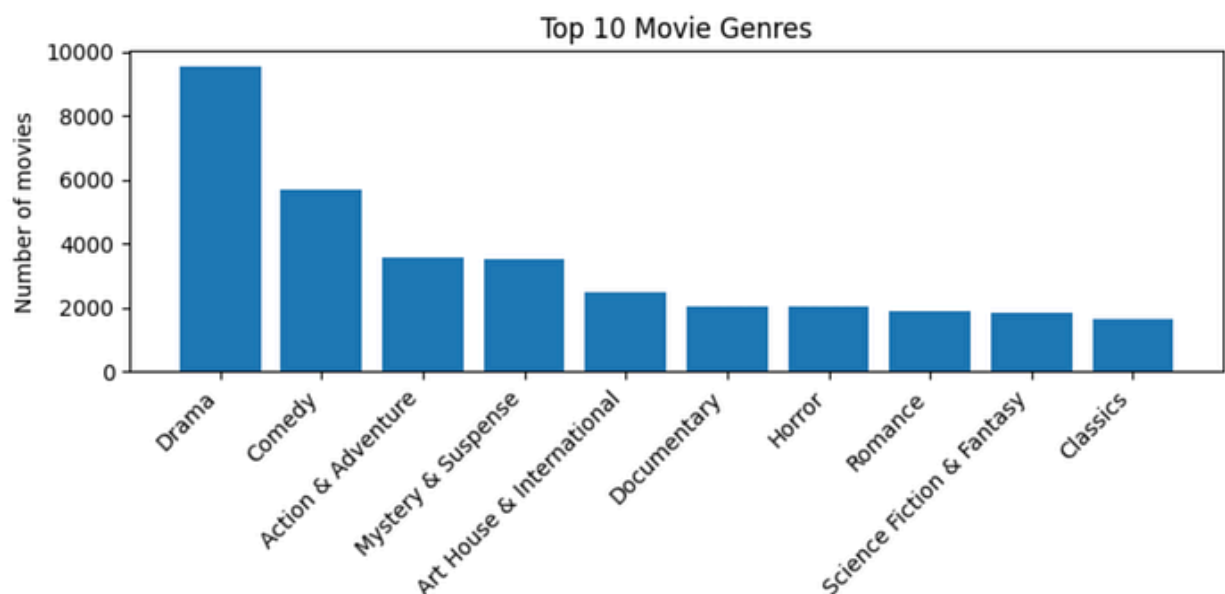
Press enter or click to view image in full size



Distribution of the top 10 book categories; Fiction dominates, with a long tail of specialized genres.

On the movie side, Drama and Comedy lead, followed by Action & Adventure, Mystery & Suspense, Art House & International, and Documentary, with smaller clusters in Horror, Romance, Science Fiction & Fantasy, and Classics. This gives us good coverage for the user profiles we care about (e.g., action/sci-fi, mystery/thriller, drama).

Press enter or click to view image in full size



Distribution of the top 10 movie genres; drama and comedy are most common, with solid representation of action, mystery, and sci-fi.

Turning Raw Reviews into a Cross-Domain “Semantic Bridge”

Our project starts from three noisy, real-world data sources: a large catalog of books with metadata (title, categories, description), a separate table of user-written book reviews and ratings, and Rotten Tomatoes movie metadata with genres, critic consensus, and aggregated ratings.

Very early on we made a deliberate choice: treat user reviews as the primary semantic signal, not publisher descriptions. Reader comments capture tone, pacing, themes, and emotional reactions in a way that short blurbs almost never do. To lean into that, we kept only books that had at least 5 reviews, to avoid learning from one-off or spammy ratings, and we capped each book at 20 reviews to keep the embedding pipeline scalable and prevent a

few blockbusters from dominating the representation. This gave us a book corpus that is both semantically rich and computationally manageable.

LLM-Aware Text Cleaning

Instead of traditional NLP preprocessing (stemming, stopwords, etc.), we optimized our pipeline for SentenceTransformer-style LLM embeddings: convert to lowercase, strip HTML and URLs, remove emojis and non-informative special characters, collapse extra whitespace, and convert numeric ratings into short phrases like “very highly rated” or “mixed reviews” so that rating signals show up in the text itself.

The key design principle was: do not distort the natural language the LLM sees. We clean aggressively for noise, but keep syntax, grammar, and sentiment intact, because those are exactly what the embedding model is good at encoding.

Building “Semantic Bridge” Fields for Movies and Books

To connect movies and books in a shared semantic space, we built composite text fields that act as a “semantic bridge” for each domain. For movies: plot summary, critic/audience consensus, genres and other metadata, and rating text (our textualized numeric ratings). For books: cleaned review text (multiple reviews per book), short review summaries (where available), categories / subjects, and rating text, plus descriptions when they add meaningful context.

We also introduced a dual ID system for books: one ID for the grouped title, and separate IDs for individual reviews. That let us aggregate reviews into a single book vector without losing traceability back to individual interactions later.

Component-Level Feature Engineering

Rather than throwing all text into one giant string, we engineered separate semantic components and embedded them

independently with all-mpnet-base-v2 SentenceTransformer. For each movie:

- **E_plot** — narrative, conflict, and story elements
- **E_consensus** — critic + audience summaries with high-level sentiment
- **E_metadata** — genres / tags / structural info
- **E_ratings** — textualized rating cues

For each book:

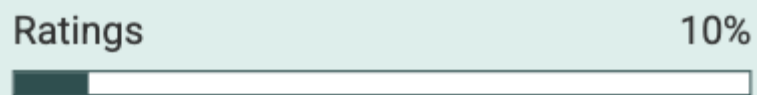
- **E_reviews** — weighted aggregation of individual review embeddings using helpfulness as weights
- **E_description** — official book description
- **E_metadata** — categories/tags
- **E_ratings** — rating text

This gives us fine-grained control later: we can weight plot vs. reviews vs. metadata, and we are not forced to trust any single noisy field.

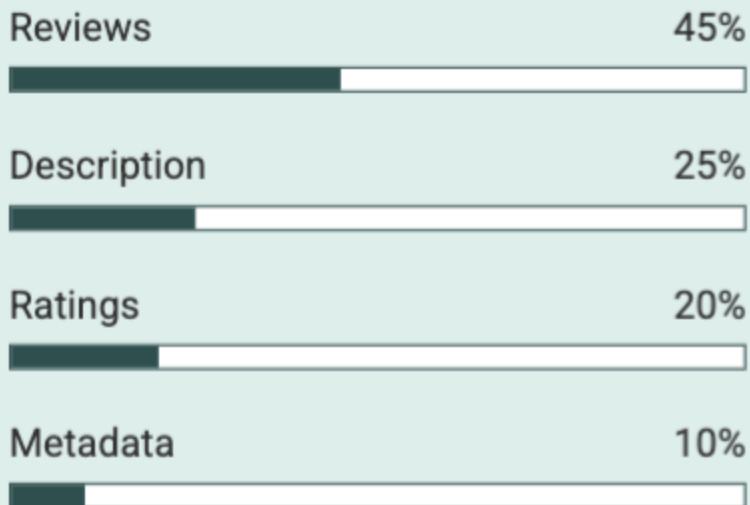
Weighted Fusion: Turning Components into Final Item Vectors

To get a single embedding per item, we combine the component vectors using a weighted average fusion.

Movies



Books



For movies: E_{plot} (0.50), $E_{\text{consensus}}$ (0.30), E_{metadata} (0.10), E_{ratings} (0.10). Here, we heavily favor the plot and critic/audience consensus, since together they capture both the core narrative (what the movie is about) and how people responded to it. Metadata and ratings still contribute as structural/contextual signals, but with lower weights so they do not dominate the semantic meaning of the movie.

For books: E_reviews (0.45), E_description (0.25), E_metadata (0.10), E_ratings (0.20). For books, user reviews are the main signal, so we let them anchor nearly half of the representation, with description and ratings filling in gaps.

The fusion is implemented via a helper like `fuse_components_weighted(...)`, which collects the available components, applies the weight dictionary, and returns a normalized `E_movie_final` or `E_book_final`. These fused vectors are saved and reused downstream so we do not recompute embeddings on every experiment.

Modeling

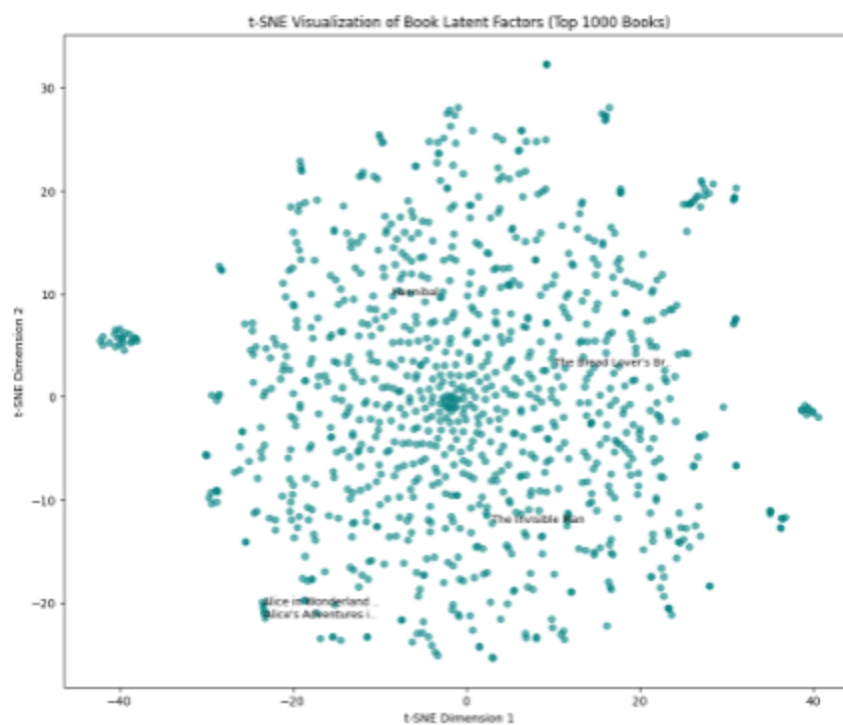
At this point, we have a shared semantic representation for movies and books. The modeling question becomes:

How much does a purely zero-shot embedding already solve cross-domain recommendation, and what do we gain (or lose) by adding a learned mapping or a collaborative baseline?

We explored three complementary model classes:

1. In-Domain Collaborative Baseline: Book NCF

As a familiar point of reference, we trained a Neural Collaborative Filtering (NCF) model that operates **only in the book domain**, using user IDs and book IDs as inputs. The architecture follows the standard GMF + MLP design: each user and book is mapped to an embedding, a dot-product style branch and a small neural network branch model their interaction, and the outputs are combined to predict how much a user will like a given book.



- **Task:** recommend books to users based only on past book ratings.
- **Evaluation:** HitRate@10 with a 1-of-99 protocol.

For each held-out (user, positive_book) pair, we sample 99 random negative books, score all 100 candidates with NCF, and check if the positive book appears in the top-10. The model recovers the correct book in the top-10 about 24% of the time on this setup, which we keep as our main in-domain baseline.

We also tried a much harsher full-catalog ranking, where the NCF has to rank the true book within the top-10 out of almost 80k titles; performance drops to essentially zero, which is expected and mainly tells us that NCF is good at beating random negatives, not at global ranking across the entire catalog.

Importantly, this model cannot do cross-domain cold-start at all — new users with only movie tastes and no book ratings are

invisible to it. That's exactly the gap our semantic models aim to fill.

2. Approach A — Zero-Shot Semantic Retrieval

Approach A asks a provocative question:

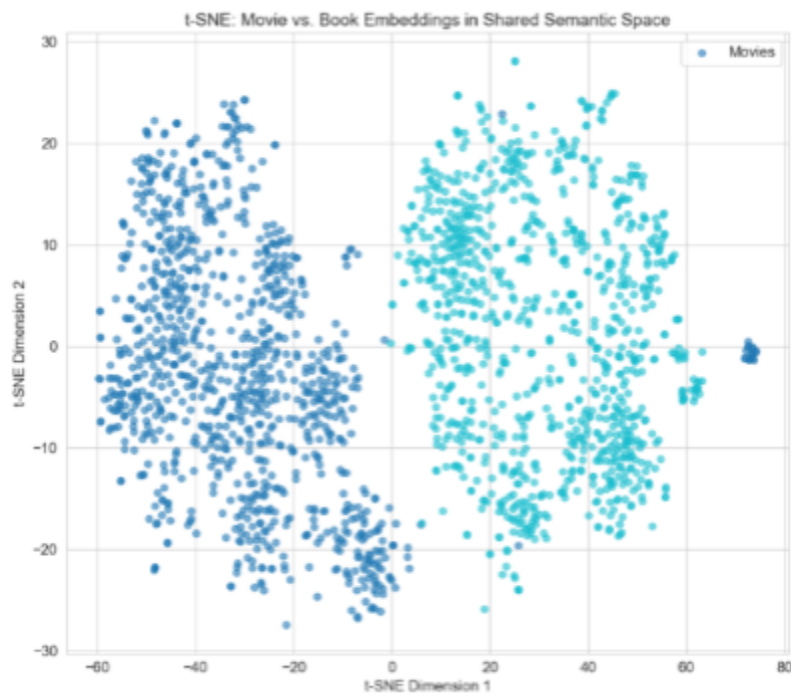
If we do nothing beyond MPNet embeddings and careful preprocessing, can we already recommend books from movies?

The pipeline is deliberately simple: use `all-mpnet-base-v2` to embed cleaned text for movies and books, then fuse and L2-normalize them to get `E_movie_final` and `E_book_final`.

Given a list of movies a user likes, retrieve their embeddings and compute a mean vector (user centroid). Normalize the centroid to get a user taste vector in the shared space. Compute cosine similarity between the user vector and every book embedding (dot product, thanks to normalization). Rank books by similarity and return the top-K as recommendations.

No additional training, no cross-domain labels, no joint rating matrix: everything comes from the LLM's semantic space plus our representation engineering.

This model serves as a cold-start recommender (it only needs a list of liked movies) and a baseline for later learned models (if we can't beat it, our mapping isn't buying us much).





t-SNE projection of movie and book embeddings before training the mapping network. The scatter plot (top) shows that items form coherent domain-specific manifolds in a shared semantic space. The density view (bottom) emphasizes how movies and books occupy related but shifted regions, motivating a small neural mapper to better align them for cross-domain recommendation.

3. Approach B — Learned Mapping (MLP + Triplet Loss)

Approach B introduces a small neural network that explicitly maps movie embeddings into book embedding space.

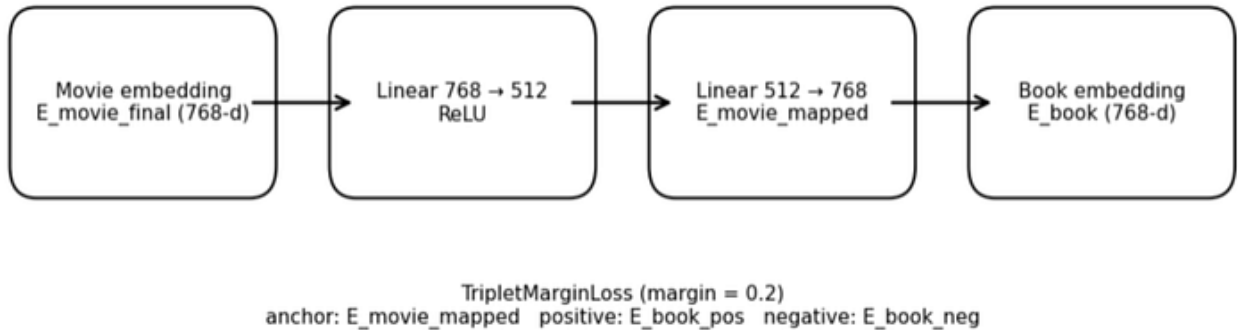
3.1 Synthetic Positives from Zero-Shot Neighbors

Because we don't have real movie→book interaction data, we generate synthetic alignment pairs. Use the zero-shot fused embeddings to compute cosine similarity between every movie and every book. For each movie, select its top-K most similar books and record (movie_id, book_id, similarity). Split these synthetic pairs into a training subset (used to form triplets) and a held-out eval subset (used later for evaluation metrics such as HitRate@10, NDCG@10, GenreMatch@1).

Each training sample becomes a triplet: Anchor (movie embedding), Positive (book embedding from the synthetic positive set for that movie), Negative (another book sampled to be less similar for that same movie).

3.2 The Mapping Architecture

Press enter or click to view image in full size



The mapping network is a lightweight MLP.

- **Input:** raw movie embedding (same dimensionality as MPNet output, e.g., 768).
- **Architecture:** Linear(input_dim $\rightarrow$ 512), ReLU, Linear(512 $\rightarrow$ output_dim), where output_dim matches the book embedding dimension.
- **Output:** mapped movie vector in book space, followed by L2 normalization for cosine similarity.

The model is trained on the synthetic training split with
Optimizer: Adam, Loss: TripletMarginLoss(margin=0.2),
Epochs: 9.

Triplet loss enforces a relative geometry: the mapped movie should be closer to its positive book than to the negative one by at least the margin. Over many triplets this shapes the embedding space so that semantically similar books and movies become neighbors. After training, we batch-map all movies, normalize the outputs, and save them for reuse.

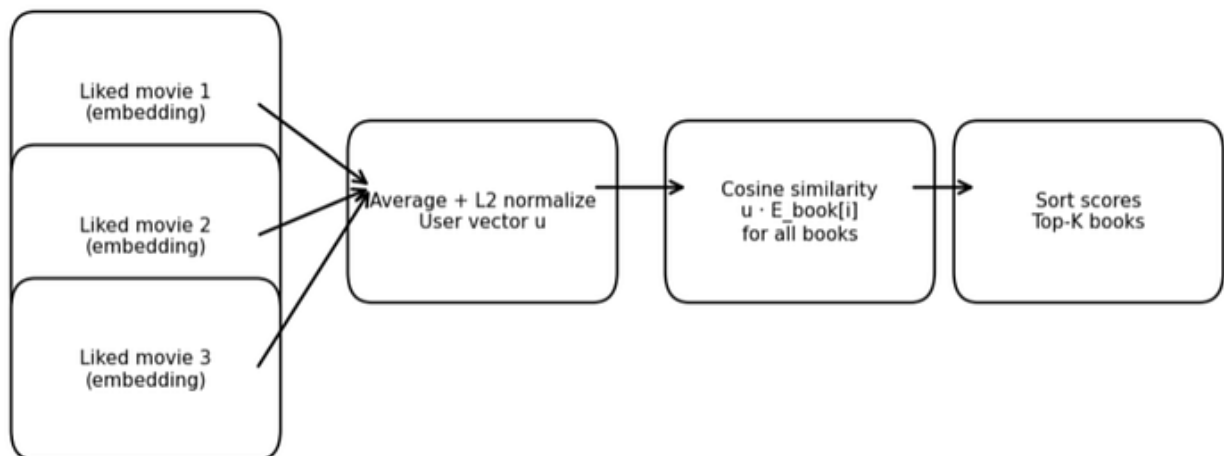
3.3 Refined Retrieval Logic

For user-level recommendations with the mapped model, we don't just rely on cosine similarity. We start from mapped user→book cosine scores, keep only the top candidate_k books by similarity, apply a category blacklist to drop unwanted types (e.g., certain juvenile or out-of-scope categories), compute a genre-aware bonus/penalty based on overlap between user

movie genres and book categories, plus hints from description text, and re-rank and deduplicate books by normalized title.

This refined retrieval pipeline is what powers the later case studies.

Press enter or click to view image in full size



For hyperparameters in this project, we intentionally kept things simple and transparent — no massive grid search, but stable defaults (MPNet backbone, moderate MLP size, 9 epochs, standard margin) so that the focus stays on the modeling ideas and evaluation, not on squeezing out a few extra decimal points.

Results

We evaluate the system along three axes: an in-domain collaborative baseline, a cross-domain system-level task, and a user-level case study.

1. In-Domain Baseline: Book NCF

On 1,000 held-out user–book interactions and a 1-of-99 ranking protocol:

HitRate@10 $\approx$ 0.24

Interpretation: When you give NCF a real user and a book they actually rated, it recovers that book in the top-10 about a quarter of the time among 100 candidates. In a full-catalog ranking (top-10 vs ~78k books), HitRate@10 drops to ~0, showing that NCF is designed to beat random candidates, not to globally sort the entire catalog.

This baseline is mainly here to anchor expectations: if we tried to force NCF into the cross-domain cold-start setting, it would fail completely, because it has no way to see movie preferences.

2. Cross-Domain System-Level Evaluation (Movies → Books)

We evaluate both semantic approaches on ~48k synthetic movie–book pairs. Each pair is a “positive” created by taking a movie and one of its nearest books in the fused embedding space. For each pair, we use the movie as a query, rank all books in the catalog (~78k) by cosine similarity, and check whether the paired book appears in the top-10.

Metrics used:

- **HitRate@10** — fraction of pairs where the correct book appears anywhere in the top-10 (“How often do we get the right book somewhere on the first page of results?”)

- **NDCG@10** — rewards putting the correct book near the top of the list rather than just barely in the top-10 (“When we’re right, do we rank the right book near the top or bury it?”)
- **Recall@10** — fraction of relevant items retrieved in the top-10; with exactly one relevant book per query, this equals HitRate@10 (“Out of what matters, how much do we actually recover in the top-10?”)
- **Precision@10** — fraction of the top-10 that is actually the target book; in our one-relevant-item setup it is just HitRate@10 divided by 10 (“Of the 10 things we show, how many are truly the one we care about?”)
- **GenreMatch@1** — probability that the top-1 recommended book shares at least one category with the target book (“Is the very first suggestion at least in the right genre neighborhood?”)

Approach A — Zero-Shot

On this synthetic task, the zero-shot baseline achieves:

- $\text{HitRate@10} \approx 0.4625$
- $\text{NDCG@10} \approx 0.2361$
- $\text{Recall@10} \approx 0.4625$
- $\text{Precision@10} \approx 0.0462$
- $\text{GenreMatch@1} \approx 0.35$

So in about 46% of synthetic pairs, the zero-shot model recovers the paired book somewhere in its top-10, using only pretrained embeddings and our fusion strategy.

To understand whether these similarities are actually meaningful, we also looked at random movie–book pairs in the same zero-shot space. For random pairs, the cosine similarity has mean ≈ 0.26 , 95th percentile ≈ 0.44 . By contrast, the aligned evaluation pairs after mapping cluster around ~ 0.56 . That means most “true” movie–book pairs live well above what random pairs

ever reach. Even before mapping, the zero-shot space clearly carries strong cross-domain signal.

Approach B — Mapped

For the same 48k evaluation pairs and full-catalog ranking protocol, the mapped model achieves:

- $\text{HitRate@10} \approx 0.3055$
- $\text{NDCG@10} \approx 0.1666$
- $\text{Recall@10} \approx 0.3055$
- $\text{Precision@10} \approx 0.0305$
- $\text{GenreMatch@1} \approx 0.4060$

Two patterns emerge. **Zero-shot wins on ranking metrics** (HitRate@10 , NDCG@10). This is not surprising: the synthetic positives were constructed from the zero-shot space, so part of the task is “Can you re-discover your own nearest neighbors?”

Mapped wins on semantic alignment. GenreMatch@1 is

higher for the mapped model (0.406 vs 0.35), so its very first recommendation is more likely to share categories with the target book.

Press enter or click to view image in full size

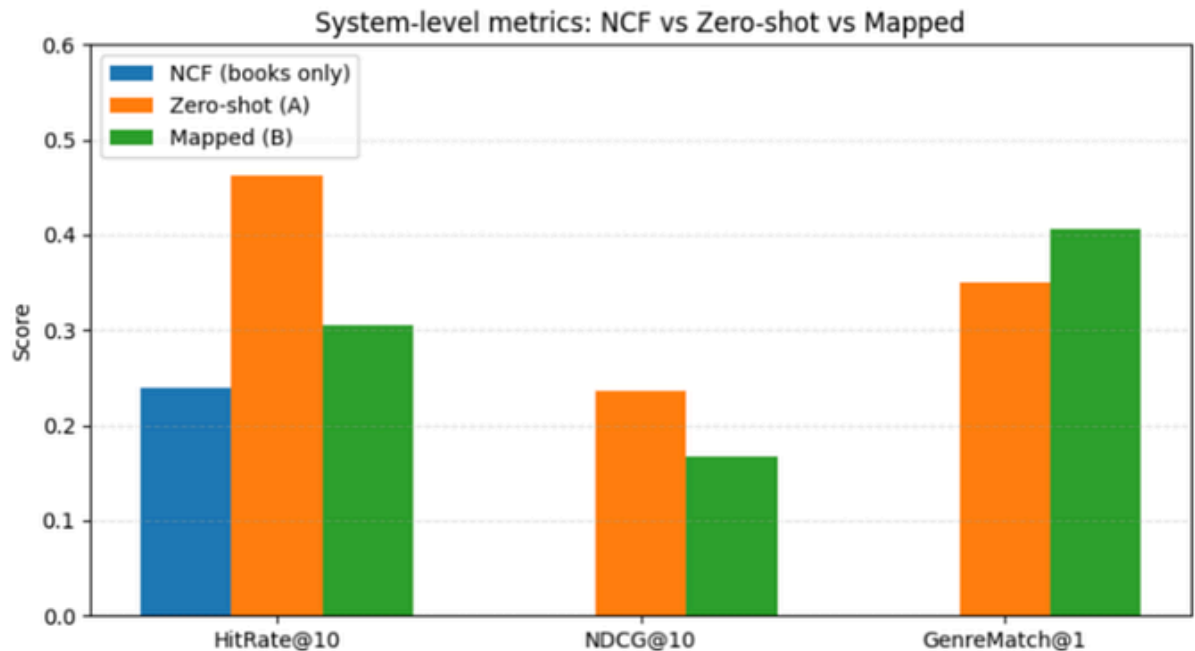
Metric	Zero Shot	Mapped
HitRate@10	0.4625 (random: 0.27)	0.3055 (random: 0.07)
NDCG@10	0.2361	0.1666
Recall@10	0.4625	0.3055
Precision@10	0.0462	0.0305
GenreMatch@1	0.35	0.4060

The random-baseline story makes this trade-off clearer. For random movie–book pairs: zero-shot space has mean cosine $\approx$

0.27, mapped space has mean cosine ≈ 0.07 . So the mapper does two things at once: it keeps paired movie–book similarities high (aligned pairs still cluster around ~ 0.56), and it pushes unrelated cross-domain pairs closer to “no relation” (near 0) instead of ~ 0.27 .

In other words, the mapper **increases the contrast between signal and noise**: aligned pairs stay close, while random pairs become much more orthogonal. Combined with the higher GenreMatch@1 and more coherent user-level recommendations, this gives additional confidence that the mapped space is capturing cleaner cross-domain semantics, even if the zero-shot space slightly outperforms it on this particular synthetic ranking benchmark.

Press enter or click to view image in full size



Takeaway

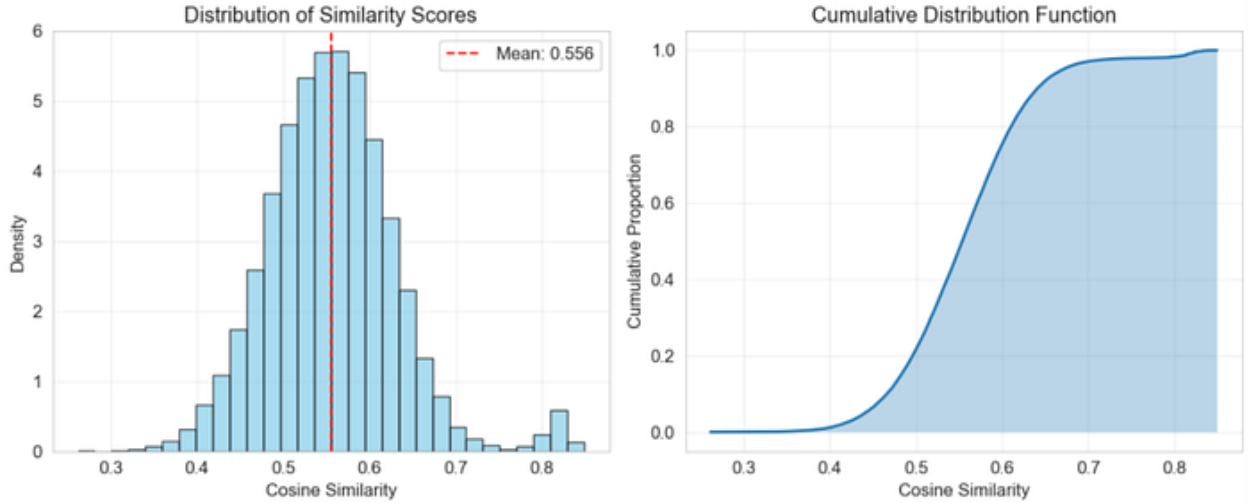
The mapping trades some raw recall for better structural alignment at the genre/category level. That is what you would expect from a contrastive model trained on synthetic alignment rather than on user clicks.

To sanity-check how well the mapping network aligns movies and books, we looked at the distribution of cosine similarities for our synthetic evaluation pairs (each pair is a movie and its matched book after mapping the movie into book space). To

understand whether these similarities are actually meaningful, we also looked at random movie–book pairs in the same zero-shot space. For random pairs, the cosine similarity has mean ≈ 0.26 , 95th percentile ≈ 0.44 . By contrast, the aligned evaluation pairs after mapping cluster around ~ 0.56 . That means most “true” movie–book pairs live well above what random pairs ever reach. Very few pairs fall below 0.4, which means that once the mapper is trained, typical aligned pairs are substantially more similar than random cross-domain pairs.

The cumulative distribution on the right tells the same story from a different angle: the curve rises steeply between ≈ 0.5 and 0.65, so a large fraction of pairs lie in that high-similarity band. In other words, for most synthetic movie–book pairs, the mapped movie embedding ends up fairly close to the corresponding book embedding in the shared space.

[Press enter or click to view image in full size](#)



Distribution of cosine similarities for aligned movie–book pairs after mapping. Most synthetic pairs concentrate around a similarity of ~ 0.56 , with a steep CDF, indicating that the mapping network consistently pulls matched movies and books close together in the shared embedding space.

3. User-Level Nolan-Style Case Study

Finally, we test the system end-to-end with a single user profile that likes “The Dark Knight”, “Inception”, “Interstellar” and 23 other similar movies. From the movie metadata we extract coarse genre tokens like: {‘science’, ‘fantasy’, ‘adventure’, ‘mystery’, ‘suspense’, ‘fiction’, ‘action’, ‘drama’}.

Using the mapped model plus refined retrieval (category blacklist + genre bonuses), we obtain top books such as crime/thriller fiction (e.g., KILLSHOT), and science-fiction and graphic novels (e.g., A Scanner Darkly, The Rock Rats: The Asteroid Wars, Superman/Aliens).

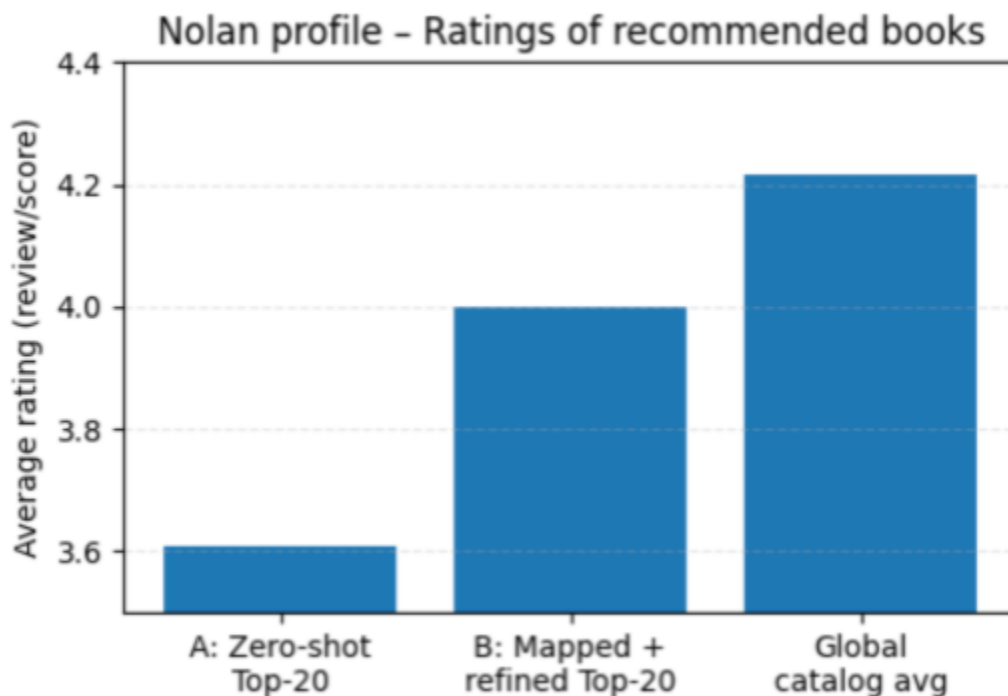
On the top-20 recommended books:

Press enter or click to view image in full size

	Approach A (Zero-Shot)	Category	Score A	Approach B (Mapped)	Category	Score B
0	Say Good-bye to Johnnie Blue	Unknown Category	0.7319	Hollywood Tough: A Shane Scully Novel	['Fiction']	0.6114
1	Last Man Standing	['Fiction']	0.6831	Paid in Blood (NCIS Series #1)	['Fiction']	0.5962
2	And Justice for One	['Fiction']	0.6792	Say Good-bye to Johnnie Blue	Unknown Category	0.5850
3	Five Star First Edition Mystery - Cracks In Th...	['Fiction']	0.6760	The Plutonium Blonde (Daw Book Collectors)	['Fiction']	0.5790
4	In Dark Places	['Fiction']	0.6745	Crashcourse	['Fiction']	0.5772
5	Hollywood Tough: A Shane Scully Novel	['Fiction']	0.6728	KILLSHOT	['Fiction']	0.5771
6	The Mediterranean Caper (Dirk Pitt Adventure S...	['Fiction']	0.6706	Five Star First Edition Mystery - Cracks In Th...	['Fiction']	0.5745
7	KILLSHOT	['Fiction']	0.6695	Into the Volcano: A Mallory & Morse Novel of E...	['Fiction']	0.5740
8	Walpurgis III (Questar Science Fiction)	['Fiction']	0.6678	A Scanner Darkly [Graphic Novel]	['Comics & Graphic Novels']	0.5724
9	Secret Justice: A Novel	['Fiction']	0.6676	Springer's Gambit: A Cole Springer Mystery	['Fiction']	0.5694

- **Genre overlap ≈ 0.70** —70% of the top-20 share at least one coarse genre token with the user's movie genres

- **Average rating $\approx 4.185 / 5$ vs global average $\approx 4.273 / 5$** — recommendations are slightly below the global mean but clearly in the well-reviewed region.



When we compare raw zero-shot vs raw mapped top-20 lists for this user, we see genre-overlap values around 0.90 vs 0.80; both are strongly genre-consistent, and the difference is within what you'd expect from user-to-user variability.

Interpretation: The mapped + refined pipeline can generate book lists that both “feel like” Nolan-style action/sci-fi/crime and remain quality-controlled by ratings. At the user level, the gap between zero-shot and mapped is small: both are capable of on-theme recommendations, and the main benefit of the mapping is more about structured control (filters, bonuses, penalties) than about pure similarity scores.

Conclusion

At the start of this project, we asked a simple question with a surprisingly complex technical backbone: Can we recommend books to someone using nothing but the movies they like? By the end, we built a full pipeline that demonstrates the answer is yes, and that modern semantic embedding models are powerful tools for solving true cold-start problems across domains.

Across data cleaning, semantic engineering, and modeling, one theme emerged repeatedly: text carries rich cross-domain structure. By treating reviews, plot summaries, categories,

consensus blurbs, and descriptive metadata as signals for meaning, and by embedding them with all-mpnet-base-v2, we created a shared space where movies and books coexist in a meaningful way. The weighted fusion approach gave us fine-grained control over what “meaning” looks like for each domain and set the stage for all downstream modeling.

Our comparisons showed that pure zero-shot retrieval (Approach A) is already a strong baseline. Without any training, it achieved a HitRate@10 near 0.46 on synthetic movie→book pairs and produced sensible, thematically coherent recommendations for real user profiles. This underscores how much structure pre-trained language models bring to cross-domain recommendation: even random movie–book pairs have moderately high cosine similarity (mean ≈ 0.27), because everything in this space “talks about stories.”

The learned mapping model (Approach B) told a more nuanced story. On the synthetic ranking metrics, it did not surpass

zero-shot, which is expected given that the evaluation pairs were constructed from the zero-shot space itself. However, it improved genre-level alignment (GenreMatch@1 from ~ 0.35 to ~ 0.41) and produced a cleaner separation between signal and noise.

For random movie–book pairs, the mean cosine similarity drops from ≈ 0.27 in the zero-shot space to ≈ 0.07 in the mapped space, while aligned pairs remain clustered around ~ 0.56 . In other words, the mapper keeps true matches close but pushes unrelated cross-domain pairs much closer to “no relation,” increasing the contrast between good and bad candidates. This, combined with more structured behavior under re-ranking and category filtering in the Nolan-style user case study, suggests that the mapped model holds stronger long-term potential, even if zero-shot currently wins on a benchmark that favors its own geometry.

The NCF book-only baseline served as an important contrast. It performed reasonably well in its intended regime (books only, 1-of-99 in-domain ranking), but its HitRate@10 collapsed when asked to rank the full catalog and it cannot be applied at all when the user arrives with only movie preferences. This reinforced the key point: collaborative filtering cannot touch the cross-domain cold-start scenario we care about, whereas semantic models can.

Taken together, these findings support our original hypothesis. LLM-based semantic embeddings can form a viable bridge between movies and books, enabling meaningful recommendations without shared users, joint training, or cross-domain ratings. Even a lightweight mapping network can meaningfully reshape this space — tightening clusters for true pairs, flattening similarity for random pairs — and a refined retrieval layer can turn raw similarities into personalized, genre-aware recommendations.

There is also substantial room for future work. Real cross-domain interaction data, if available, could train more robust mappings. A contrastive dual-encoder trained end-to-end might align movie and book spaces more faithfully. FAISS-based indexing could enable instant retrieval over hundreds of thousands of items. Richer re-ranking layers could incorporate sentiment, pacing, writing style, or even multimodal cues such as trailers and cover art.

For now, we leave with a simple takeaway: **text is a powerful modality**, and with the right preprocessing, embedding strategy, and evaluation framework, it can unlock cross-domain recommendation systems that feel intuitive and surprisingly human — even when they have never seen a single user who liked both a movie and a book.

References

[1] L. Wu, et al., “A Survey on Large Language Models for Recommendation,” arXiv preprint arXiv:2305.19860, 2023.

[2] S. Geng, et al., “Recommendation as Language Processing (P5): A Unified Pretrain, Personalized Prompt and Predict Paradigm,” in Proc. RecSys ’22, 2022.

[3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proc. EMNLP, 2019.

[4] T. Man, H. Shen, X. Jin, and X. Cheng, “Cross-Domain Recommendation: An Embedding and Mapping Approach,” in Proc. IJCAI, 2017